Population based Methods:-

* Evolved by nature.
* N Candiate Solutions ← Start

1. Population ('N')
2. Selection ('P') & "repetitions"
3. Crossover

'N'

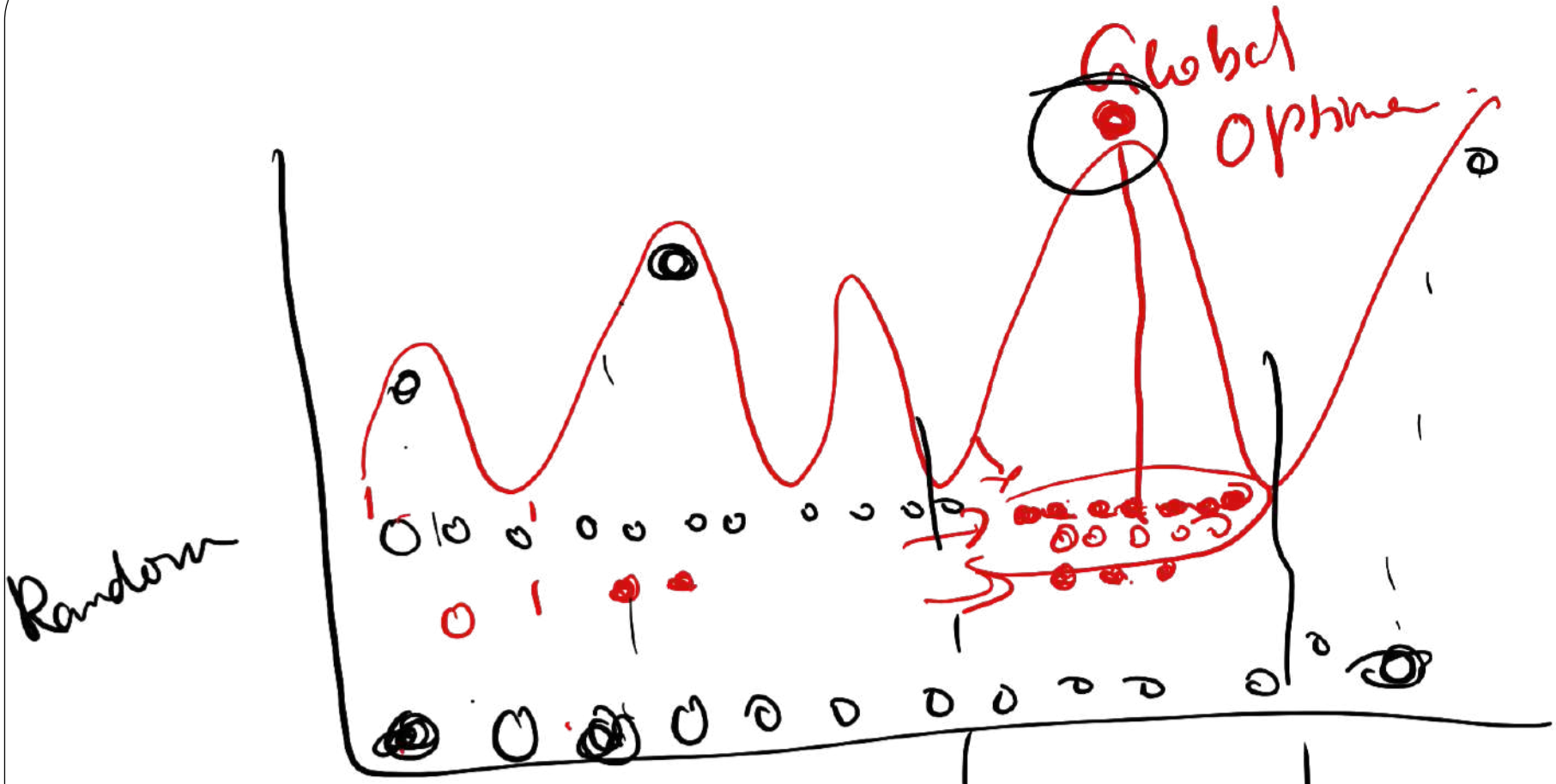
$(a \vee \neg b \vee c)$

1 (or) 0 2^3

011
100
101

GENETIC-ALGORITHM()

```
P ← create N candidate solutions        ▷ initial population
repeat
    compute fitness value for each member of P
    S ← with probability proportional to fitness value,
        randomly select N members from P
    offspring ← partition S into two halves, and randomly mate
                and crossover members to generate N offsprings
    with a low probability mutate some offsprings
    replace k weakest members of P with k strongest offsprings
until some termination criteria
return the best member of P
```



Population plays a major role

Adversarial Search:-

① In a multiagent environment {referred as game}, the impact of each agent on others is significant.

② All the agents are rational, deterministic.

③ The utility values at end of each game are always equal and opposite. → payoff {+1, −1, 0} {W, L, D}.

④ The state of a game is easy to represent, and agents are restricted to a small number of actions.

A game can be formally defined as:

① Initial State ← Common for both agents.

② Player(s) ← Alternate moves will be given to the players.

Actions(s) ←

Result(s, a) ←

Terminal_Test(s) ← +1, −1, 0

Utility Function – Evaluation function / Pay off
– Find numeric value in a game that ends in terminal state 's' for a player.

Zero-Sum Game:-

Utility(s, a) ← payoff.

→ After several movement of states, in a certain instance of game, the utility function would be.

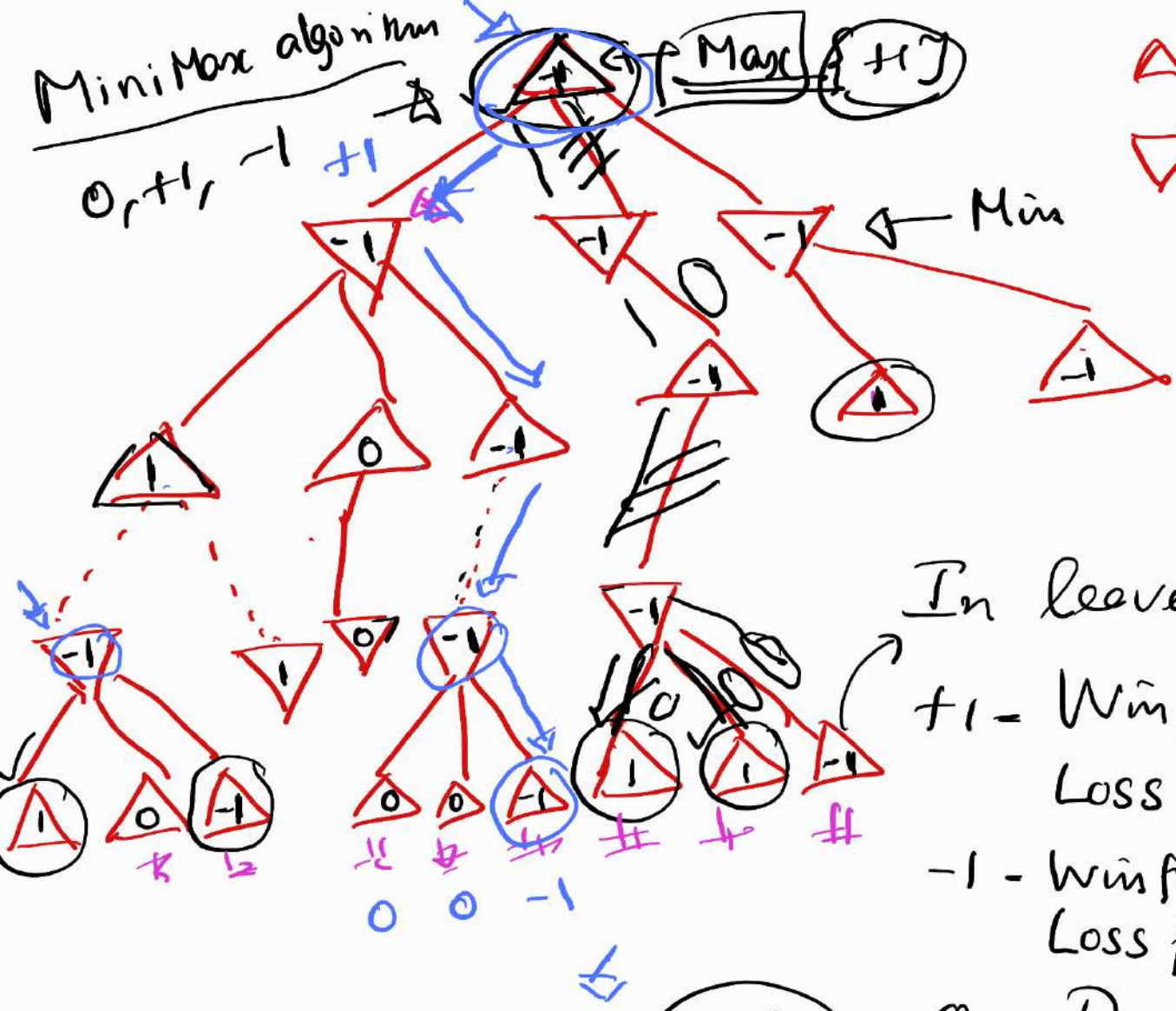
$$\left\{\begin{array}{l} 0+1 \\ 1+0 \\ \frac{1}{2}+\frac{1}{2} \end{array}\right\} \begin{array}{l} =1 \\ =1 \\ =1 \end{array}$$

MiniMax algorithm

0, +1, −1

△ - Player - 1

▽ - Player - 2

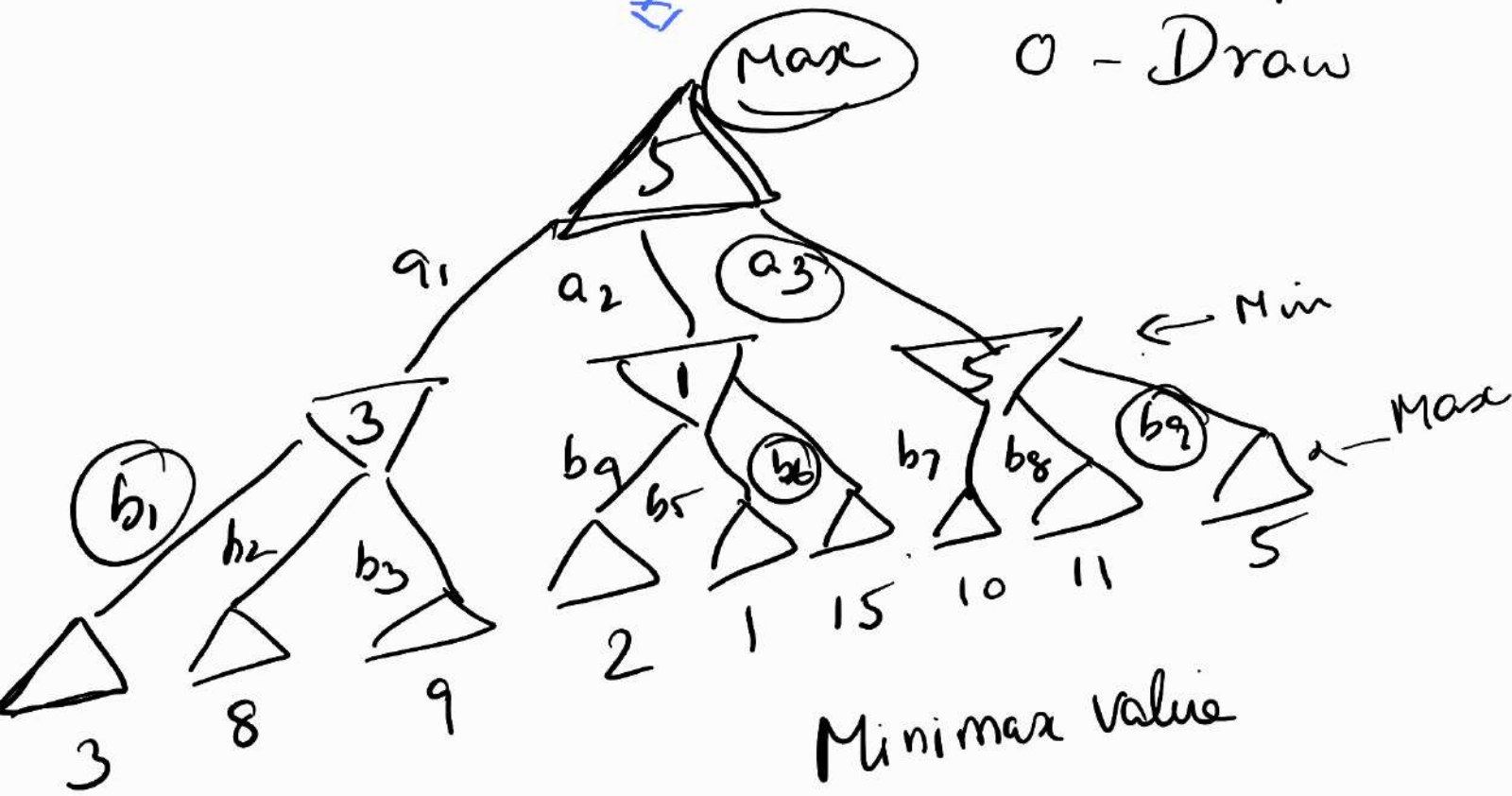


In leaves,

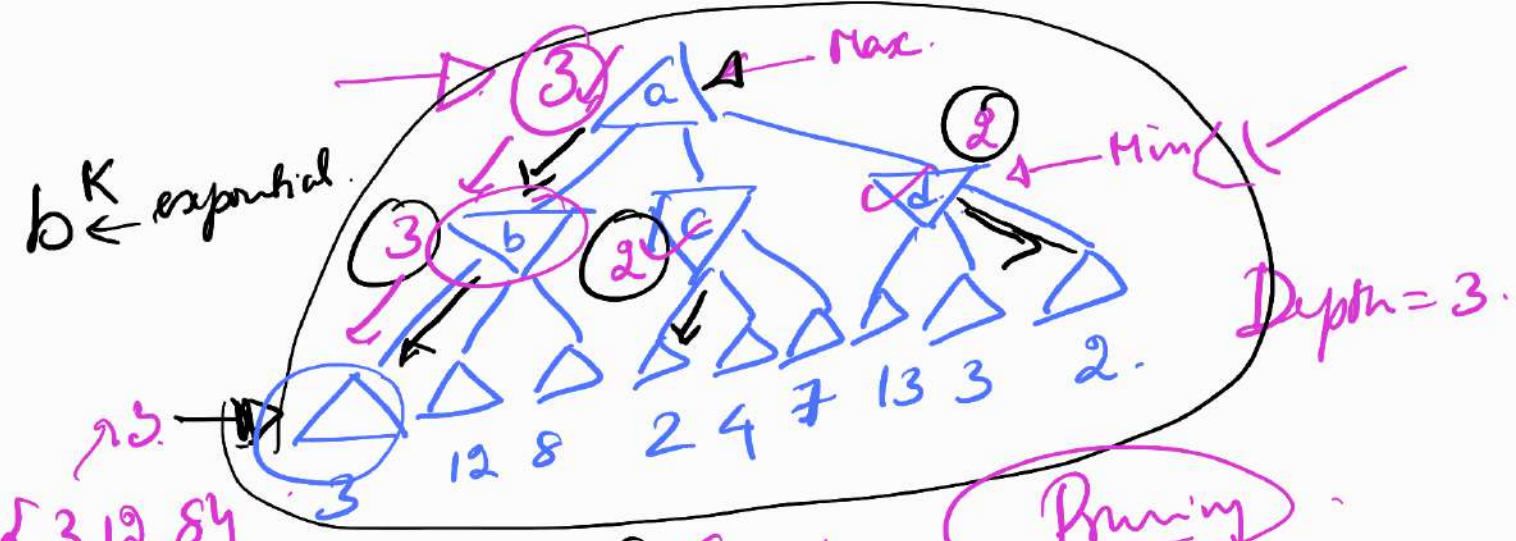
+1 - Win for Max
Loss for Min

−1 - Win for Min
Loss for Max

0 - Draw



Minimax value



$b^K \leftarrow$ exponential.

3.

{3, 2, 8}

Minimax backup use.

{ Greedy.
Depth first exploration.

Pruning.

Complexity :- branching factor → b (3). b^K ← Time.
depth → K. b.K ← Space

K - depth tree (K - ply Tree) :-

Minimax (S).

If B is a terminal node. // Base Case

return h(n) $\{+1, -1, 0\}$

else if B is a Max node.

Value ← $-\infty$

for each child C in B.

Value ← max (value, Minimax (c))

else if B is a Min node.

Value → ∞

for each child C in B.

value ← min (value, minimax(c))

Alpha-Beta Pruning :-

Alpha → Max nodes.

Beta → Min nodes.

Minimax Backup